Pointer

NRP	: 1. 3223600044
	2. 3223600055
	3. 3223600058
Nama	: 1. Devano Auriza Khairindra Heriyanto
	2. Arimbi Kusuma Jati
	3. Ihsanta Zaki Sanjaya
Kelas	: 2 D4 Teknik Komputer B
Matakuliah	: Praktikum Organisasi Mesin dan Bahasa Assembly
Tanggal	: Kamis, 5 September 2024
Dosen	: Dr. Setiawardhana ST, MT

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA 2024

POINTER

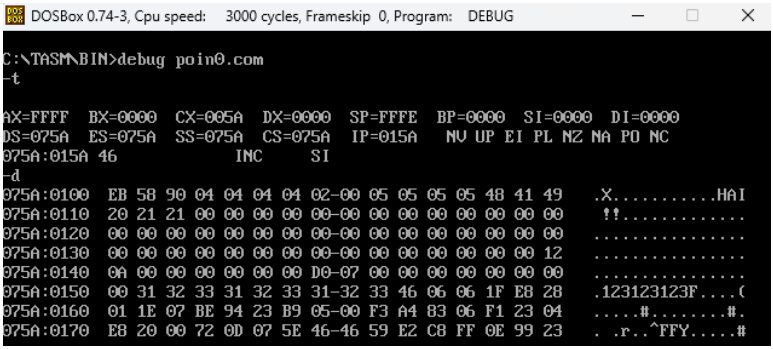
PROGRAM PENAMBAHAN DAN DEBUG

Berikut merupakan sintax dari program yang dapat menyimpan data dengan berbagai tipe data yang bedbeda-beda.

```
          .MODEL SMALL
          .CODE
          ORG 100h
TData :
             JMP Proses
             A   DB  4           ; 1 byte, nilai
awal='4'
             B   DB  4,4,4,2,?   ; 1*5 byte, nilai
awal=4,4,4,2,?
             C   DB  4 DUP(5)    ; 1*4 byte, nilai awal='5'
             D   DB  'HAI !!'    ; 6 byte berisi 6 karakter
             E   DW  ?           ; 1 word tidak diketahui
isinya
             F   DW  ?,?,?       ; 3 word tidak diketahui
isinya
             G   DW  10 DUP(?)   ;10 word tidak diketahui
isinya
             H   DD  ?           ; 1 DoubleWord tanpa nilai
awal
             I   DF  ?,?         ; 2 FarWord tanpa nilai awal
             J   DQ  0A12h       ; 1 QuadWord, nilai
awal='0A12'
             K   DT  25*80       ; 1 TenBytes, nilai
awal='2000'
             L   EQU 666         ; Konstanta, L=666
             M   DB  '123'         ; String '123'
             N   DB  '1','2','3'   ; String '123'
             O   DB   49,50,51     ; String '123'
Proses :     ;
             ;
```

```
                ;
END         Tdata
```

Apabila program dengan sintax di atas di-debug maka akan menampilkan hasil seperti di bawah ini.



Gambar 1 Hasil debug program Tipe data pada DOSBox

Analisa pada program tipe data ini menunjukkan bahwa program ini ditulis dalam assembly dengan model memori SMALL, yang artinya program ini hanya menggunakan satu segmen untuk kode dan satu segmen untuk data. Program dimulai pada offset 100h dan terdapat deklarasi berbagai variabel dengan tipe data dan nilai yang beragam. Sebagai contoh, variabel A dideklarasikan sebagai byte (DB) dengan nilai awal 4, sementara B dideklarasikan sebagai array yang berisi 5 byte dengan nilai 4, 4, 4, 2, dan nilai terakhir tidak diinisialisasi. Variabel C menggunakan deklarasi DUP untuk menduplikasi nilai 5 sebanyak empat kali, sehingga menghasilkan 4 byte dengan nilai yang sama. Selain itu, ada variabel D yang menyimpan string 'HAI !!', di mana setiap karakter string disimpan sebagai satu byte, sehingga totalnya ada 6 byte. Variabel E dan F merupakan deklarasi tipe word tanpa nilai awal, dengan E sebagai satu word dan F terdiri dari tiga word. Sedangkan G dideklarasikan sebagai sepuluh word yang juga belum diinisialisasi.

Selain word dan byte, program ini juga menggunakan double word dan quad word, seperti yang terlihat pada variabel H, yang merupakan sebuah double word tanpa nilai awal, dan J, yang dideklarasikan sebagai quad word dengan nilai awal 0A12h. Ada juga penggunaan far word pada variabel I, yang terdiri dari dua far word, masing-masing tanpa nilai awal. Variabel K menggunakan tipe data ten-byte dengan nilai 25*80, yang berarti nilai 80h disimpan sebanyak 25 kali. Program ini juga mencakup sebuah konstanta, L, yang didefinisikan dengan nilai 666 menggunakan instruksi EQU, yang berarti nilainya tidak akan berubah selama program berjalan. Variabel lainnya seperti M, N, dan O merepresentasikan string '123', namun dengan cara yang berbeda: M menyimpan string '123' sebagai satu kesatuan, N menyimpan setiap karakter '1', '2', dan '3' sebagai byte terpisah, sementara O menyimpan nilai ASCII dari karakter '1', '2', dan '3' secara langsung.

.

BHMESIN.ASM

Berikut merupakan sintax dari program BHMESIN.ASM:

```
;=====================================;
;       PROGRAM : BHSMESIN.ASM        ;
;       AUTHOR  : S'to                ;
;       FUNGSI  : MEMBUNYIKAN SPEAKER ;
;                 DENGAN DATA PROGRAM ;
;                                     ;
;=====================================
      .MODEL SMALL
      .CODE
     ORG 100h
Tdata:DB 0E4h,61h,24h,0FEh,0E6h,61h,0B9h,0D0h,7h,0BBh,9Ah
      DB 2h,8Bh,0D1h,51h,34h,2h,0E6h,61h,0D1h,0C3h,73h,6h
      DB 83h,0C1h,0h,0EBh,0Bh,90h,52h,2Bh,0D1h,87h,0D1h,5Ah
      DB 81h,0C1h,34h,8h,0E2h,0FEh,59h,0E2h,0E2h,0CDh,20h
END   Tdata
```

Apabila program dengan sintax di atas di-debug maka akan menampilkan hasil seperti di bawah ini.

```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG
-r
AX=FFFF  BX=0000  CX=002E  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0100   NV UP EI PL NZ NA PO NC
075A:0100 E461          IN      AL,61
-t

AX=FF00  BX=0000  CX=002E  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0102   NV UP EI PL NZ NA PO NC
075A:0102 24FE          AND     AL,FE
-t

AX=FF00  BX=0000  CX=002E  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0104   NV UP EI PL ZR NA PE NC
075A:0104 E661          OUT     61,AL
-t

AX=FF00  BX=0000  CX=002E  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0106   NV UP EI PL ZR NA PE NC
075A:0106 B9D007        MOV     CX,07D0
-t

AX=FF00  BX=0000  CX=07D0  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0109   NV UP EI PL ZR NA PE NC
075A:0109 BB9A02        MOV     BX,029A
-
```

Gambar 2 Hasil debug program pengurangan pada DOSBox

```
C:\TASM\BIN>poin

C:\TASM\BIN>S_
```

Gambar 3 Untuk mengeluarkan suara, ketikkan nama program

Analisa pada program bhmesin.asm ini menunjukkan bahwa program dapat membunyikan speaker dengan menggunakan data dari program itu sendiri. Segmen data dalam program ini berisi serangkaian nilai byte yang dinyatakan dalam bentuk heksadesimal, seperti 0E4h, 61h, 24h, dan seterusnya. Byte-byte ini kemungkinan merupakan instruksi mesin x86, yang dieksekusi oleh CPU untuk melakukan operasi yang diinginkan, termasuk manipulasi register, aritmetika, serta kontrol alur. Salah satu byte yang menonjol adalah CD 20h, yang merupakan interrupt DOS 20h yang digunakan untuk mengakhiri program dan kembali ke sistem operasi DOS, yang menandai bahwa program ini berakhir setelah serangkaian instruksi dijalankan.

Fungsi utama dari program ini adalah untuk memanfaatkan speaker komputer dengan menggunakan data program tersebut. Namun, jika diperhatikan lebih lanjut, program ini tidak memiliki bagian kode yang secara eksplisit mengontrol speaker, tetapi lebih terlihat memanipulasi register dan mungkin menyiapkan kondisi untuk membunyikan speaker di bagian lain program atau melalui penggunaan instruksi tertentu yang terkompilasi dari data byte tersebut. Beberapa instruksi yang dapat diidentifikasi dari segmen data termasuk 0E4h yang mungkin merupakan instruksi IN untuk input data dari port, serta 0D1h, yang merupakan instruksi aritmetika untuk penggeseran bit. Di sisi lain, instruksi seperti 0EBh menunjukkan adanya lompatan (jump) dalam alur eksekusi program.

Program ini hanya menyimpan data heksadesimal tanpa deklarasi eksplisit dari bagian kode yang memproses instruksi tersebut. Ini dapat diartikan bahwa byte-byte di segmen data mungkin sudah disiapkan sebagai urutan instruksi mesin yang akan dieksekusi langsung oleh prosesor saat program berjalan. Selain itu, ORG 100h menandakan bahwa program ini akan dijalankan dalam mode COM di DOS, di mana titik awal eksekusi dimulai pada offset 100h di dalam memori. Segmen kode ini juga menampilkan akhir program dengan instruksi END Tdata, yang memberitahu assembler bahwa eksekusi akan berakhir pada segmen data yang telah dideklarasikan.

PROGRAM PENYIMPAN DATA DALAM MEMORY

Berikut merupakan sintax dari program yang dapat menyimpan data dalam memory:

```
            .MODEL SMALL
            .CODE
            ORG 100h

TData :   JMP Proses
              A    DB   12h,34h
              B    DW   0ABCDh
              C    DD   56789018h
              D    DB   40 DUP(1)

END      Tdata
```

Apabila program dengan sintax di atas di-debug maka akan menampilkan hasil seperti di bawah ini.

```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG

C:\TASM\BIN>debug poin2.asm
-d
073F:0100  0D 0A 20 20 20 20 20 20-20 20 20 20 2E 4D 4F 44   ..           .MOD
073F:0110  45 4C 20 53 4D 41 4C 4C-20 0D 0A 20 20 20 20 20   EL SMALL ..
073F:0120  20 20 20 20 20 2E 43 4F-44 45 20 0D 0A 20 20 20        .CODE ..
073F:0130  20 20 20 20 20 20 20 4F-52 47 20 31 30 30 68 20          ORG 100h
073F:0140  0D 0A 20 0D 0A 54 44 61-74 61 20 3A 20 20 4A 4D   .. ..TData :  JM
073F:0150  50 20 50 72 6F 73 65 73-20 0D 0A 20 20 20 20 20   P Proses ..
073F:0160  20 20 20 20 20 20 20 20-41 20 20 20 44 42 20 20           A   DB
073F:0170  31 32 68 2C 33 34 68 20-0D 0A 20 20 20 20 20 20   12h,34h ..
-d
073F:0180  20 20 20 20 20 20 20 42-20 20 20 44 57 20 20 30           B   DW  0
073F:0190  41 42 43 44 68 20 20 20-20 20 20 20 20 20 20 20   ABCDh
073F:01A0  20 0D 0A 20 20 20 20 20-20 20 20 20 20 20 20 20    ..
073F:01B0  43 20 20 20 44 44 20 20-35 36 37 38 39 30 31 38   C   DD  56789018
073F:01C0  68 20 0D 0A 20 20 20 20-20 20 20 20 20 20 20 20   h ..
073F:01D0  20 44 20 20 20 44 42 20-20 34 30 20 44 55 50 28    D   DB  40 DUP(
073F:01E0  31 29 20 20 20 20 20 20-0D 0A 20 0D 0A 45 4E 44   1)       .. ..END
073F:01F0  20 20 20 20 20 54 64 61-74 61 20 00 00 00 00 00        Tdata .....
```

Gambar 4 Hasil debug program penyimpanan data pada DOSBox

Di dalam segmen data program ini, terdapat beberapa variabel yang dideklarasikan dengan berbagai ukuran dan nilai. Variabel A menyimpan dua byte dengan nilai heksadesimal 12h dan 34h. Ini berarti variabel ini memuat data 2 byte secara berurutan. Variabel B menyimpan satu word (2 byte) dengan nilai heksadesimal 0ABCDh, yang berarti ia menyimpan data 2 byte yang lebih besar dalam bentuk nilai heksadesimal tersebut.

Variabel C dideklarasikan sebagai double word (4 byte) dengan nilai heksadesimal 56789018h, menyimpan data 4 byte sekaligus. Ini adalah nilai yang lebih besar dan dapat digunakan untuk menyimpan informasi lebih besar dalam satu unit data. Terakhir, variabel D menggunakan DUP untuk menginisialisasi 40 byte dengan nilai 1, yang berarti setiap byte dalam rentang ini akan memiliki nilai 1, sehingga menyediakan blok memori yang konsisten untuk keperluan penyimpanan data seragam.

PROGRAM POINTER

Berikut merupakan sintax dari program pointer yang dapat emindahkan data dari variable maupun register yang berbeda tipe datanya dengan pointer:

```
;==================================;
;   PROGRAM : PTR.ASM              ;
;   AUTHOR  : S'to                 ;
;   FUNGSI  : MEMINDAHKAN DATA     ;
;             ANTAR TIPE DATA YANG;
;             BERBEDA !!!          ;
;==================================;
          .MODEL SMALL
          .CODE
          ORG 100h
TData :
Proses:
        JMP   Proses            ; Lompat ke Proses
          A     DW  01EFh       ; 2 Byte
          B     DW  02FEh       ; 2 Byte
          D     DD  ?           ; 4 Byte
        MOV  AL,BYTE PTR A      ; AL=EF, AX=?EF
        MOV  AH,BYTE PTR A+1    ; AH=01, AX=01EF
        MOV  BX,B               ; BX=02FE
        MOV  WORD PTR D,AX      ; D=??01EF
        MOV  WORD PTR D+2,BX    ; D=02FE01EF
        INT  20h                ; Kembali ke DOS
END     TData
```

Apabila program dengan sintax di atas di-debug maka akan menampilkan hasil seperti di bawah ini.

```
AX=FFFF  BX=0000  CX=0024  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=010B   NV UP EI PL NZ NA PO NC
075A:010B 2E            CS:
075A:010C A00301        MOV     AL,[0103]                          CS:0103=EF
-t

AX=FFEF  BX=0000  CX=0024  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=010F   NV UP EI PL NZ NA PO NC
075A:010F 2E            CS:
075A:0110 8A260401      MOV     AH,[0104]                          CS:0104=01
-t

AX=01EF  BX=0000  CX=0024  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0114   NV UP EI PL NZ NA PO NC
075A:0114 2E            CS:
075A:0115 8B1E0501      MOV     BX,[0105]                          CS:0105=02FE
-t

AX=01EF  BX=02FE  CX=0024  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0119   NV UP EI PL NZ NA PO NC
075A:0119 2E            CS:
075A:011A A30701        MOV     [0107],AX                          CS:0107=0000
```

Gambar 5 Hasil debug program pointer pada DOS Box

Pada program pointer ini, berisi serangkaian instruksi untuk memindahkan data antar variabel dengan tipe data yang berbeda. Variabel A dideklarasikan sebagai word (2 byte) dengan nilai 01EFh, dan variabel B juga dideklarasikan sebagai word dengan nilai 02FEh. Variabel D dideklarasikan sebagai double word (4 byte) tanpa nilai awal. Di dalam label Proses, beberapa instruksi kunci dijalankan untuk memanipulasi data. Pertama, MOV AL,BYTE PTR A mengambil byte pertama dari A dan memindahkannya ke register AL, sehingga AL berisi nilai EFh. Kemudian, MOV AH,BYTE PTR A+1 mengambil byte kedua dari A dan memindahkannya ke register AH, sehingga AH berisi nilai 01h. Gabungan AL dan AH membentuk AX dengan nilai 01EFh.

Selanjutnya, MOV BX,B memindahkan nilai dari B (yaitu 02FEh) ke register BX. Instruksi berikutnya, MOV WORD PTR D,AX, menyimpan nilai AX (01EFh) ke bagian awal dari variabel D, sehingga D diinisialisasi dengan 01EFh pada byte pertama dan kedua. Instruksi MOV WORD PTR D+2,BX kemudian menyimpan nilai BX (02FEh) ke byte ketiga dan keempat dari D, sehingga D akhirnya berisi 02FE01EFh. Program ini berakhir dengan instruksi INT 20h, yang mengakhiri program dan kembali ke DOS.

PROGRAM KALIPTR.ASM

Berikut merupakan program yang dapat memindahkan data dari 2 register 16 bit menuju 1 variabel 32 bit:

```
;   PROGRAM : KALIPTR.ASM        ;
;   AUTHOR  : S'to               ;
;   FUNGSI  : MENGALIKAN BILANGAN;
;             16 BIT, HASIL      ;
;             PADA DX:AX         ;
;==============================;
         .MODEL SMALL
         .CODE
         ORG 100h
TData :
Proses:
        JMP   Proses            ; Lompat ke Proses
        A     DW  01EFh       ; 2 Byte
        B     DW  02FEh       ; 2 Byte
        Hsl   DD  ?           ; 4 Byte
        MOV  AX,A               ; AX=1EF
        MUL  B                  ; Kalikan 1FH*2FE
        MOV  WORD PTR Hsl,AX    ; AX bernilai C922, Hsl=??C922
        MOV  WORD PTR Hsl+2,DX ; DX bernilai 0005, Hsl=0005C922
        INT 20h                 ; Kembali ke DOS
END     TData
```

Apabila program dengan sintax di atas di-debug maka akan menampilkan hasil seperti di bawah ini.

```
-t

AX=C922  BX=0000  CX=001F  DX=0005  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0114   OV UP EI PL NZ NA PO CY
075A:0114 2E            CS:
075A:0115 A30701        MOV     [0107],AX                           CS:0107=0000
-t

AX=C922  BX=0000  CX=001F  DX=0005  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0118   OV UP EI PL NZ NA PO CY
075A:0118 2E            CS:
075A:0119 89160901      MOV     [0109],DX                           CS:0109=0000
-t

AX=C922  BX=0000  CX=001F  DX=0005  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=011D   OV UP EI PL NZ NA PO CY
075A:011D CD20          INT     20
```

Gambar 10 Hasil debug program kaliptr.asm dengan DOS Box

Program ini berfungsi untuk mengalikan dua bilangan 16-bit dan menyimpan hasilnya dalam format 32-bit. Setelah melompat ke label Proses, program melakukan beberapa operasi aritmatika yang penting. Variabel A menyimpan nilai 01EFh dan variabel B menyimpan nilai 02FEh. Program memuat nilai dari A ke register AX, sehingga AX berisi 01EFh. Kemudian, instruksi MUL B digunakan untuk mengalikan nilai dalam AX dengan nilai B. Instruksi MUL secara otomatis mengalikan AX dengan B, menghasilkan hasil 32-bit yang disimpan di register DX . Untuk bilangan 16-bit, hasil perkalian ini ditempatkan dalam DX (high word) dan AX (low word).

Hasil dari perkalian ini adalah C922h di AX dan 0005h di DX. Oleh karena itu, hasil 32-bit dari perkalian disimpan dalam Hsl dengan nilai 0005C922h. Instruksi MOV WORD PTR Hsl,AX menyimpan nilai AX (C922h) ke dua byte pertama dari variabel Hsl, dan instruksi MOV WORD PTR Hsl+2,DX menyimpan nilai DX (0005h) ke dua byte terakhir dari Hsl. Sehingga, Hsl akhirnya berisi hasil perkalian lengkap 0005C922h`.

KESIMPULAN

Pada praktikum ini, kami berhasil mengimplementasikan berbagai program assembly dengan berbagai fungsi dan Teknik manipulasi data.

1. Tipe Data, memperlihatkan deklarasi dan inisialisasi berbagai tipe data dalam memori, seperti byte, word, dan string.
2. BHMESIN.ASM, memanfaatkan data heksadesimal untuk menghasilkan suara melalui speaker komputer, meski tidak ada kontrol langsung untuk speaker dalam kode.
3. Penyimpanan Data dalam Memori, menyimpan data dengan ukuran yang berbeda, seperti byte, word, dan double word, serta menggunakan DUP untuk menginisialisasi blok memori.
4. Pointer, menunjukkan cara memindahkan data antar variabel dengan tipe yang berbeda menggunakan register dan pointer.
5. KALIPTR.ASM, mengalikan dua bilangan 16-bit dan menyimpan hasilnya dalam format 32-bit, dengan hasil akhir yang disimpan dalam variabel 32-bit.

Praktikum ini secara keseluruhan memberikan pemahaman yang jelas tentang bagaimana pointer bekerja dan pengaplikasiannya dalam assembly.